



17. Neural Networks for Texts

Teaching Machines to Read, One Token at a Time.

Previous Section:

 [16. Recurrent Neural Networks](#)

Contents:

[1. Introduction to Textual Data](#)

[1.1. Standardization](#)

[1.2. Tokenization](#)

[1.3. Vocabulary and Indexing](#)

[1.4. Text Vectorization](#)

[1.5. Encoding and Decoding](#)

[2. Representing Groups of Words](#)

[2.1. Bag of Words \(BoW\)](#)

[2.2. TF-IDF Weighting](#)

[2.3. Sequence Models](#)

[2.4. Word Embeddings](#)

[2.5. Padding and Masking](#)

[3. Implementation using Python](#)

[3.1. Loading the IMDB Dataset](#)

[3.2. Text Standardization and Tokenization](#)

[3.3. Building a Vocabulary and Encoding Text](#)

[3.4. TensorFlow's TextVectorization Layer](#)

[3.5. Building a Bag-of-Words Sentiment Classifier](#)

[3.6. Sequence Representation with Embeddings](#)

[3.7. Positional Embeddings](#)

[3.8. Looking Ahead: Transformers and Seq2Seq Models](#)

[Summary](#)

[References](#)

One of the most transformative achievements of deep learning is that computers are no longer limited to numbers, images, or structured tables—they can now understand and generate human language. Whether it is translating between languages, answering questions, summarizing documents, or powering modern chatbots, neural networks have fundamentally changed how machines process text.

Unlike programming languages, natural languages were not designed with strict rules.

- Programming languages define grammar first and require programmers to follow it precisely.
- Human languages evolved in the opposite direction: people began speaking first, and grammar was only formalized afterward.
- As a result, natural language is often ambiguous, inconsistent, and constantly evolving across cultures and time.

The goal of **Natural Language Processing (NLP)** is to develop algorithms capable of processing words, sentences, and entire documents while extracting meaningful patterns from them. Modern NLP models are largely statistical learners. Just as computer vision models discover patterns from pixels, NLP models discover patterns from sequences of words and sentences.

Before deep learning, NLP relied heavily on handcrafted features created by domain experts.

- Engineers manually designed rules to identify keywords, grammatical structures, or linguistic patterns.
- Modern deep learning shifted the focus from manually creating features to automatically **learning representations** through techniques such as **word embeddings, recurrent neural networks (RNNs), attention mechanisms, and Transformers**.

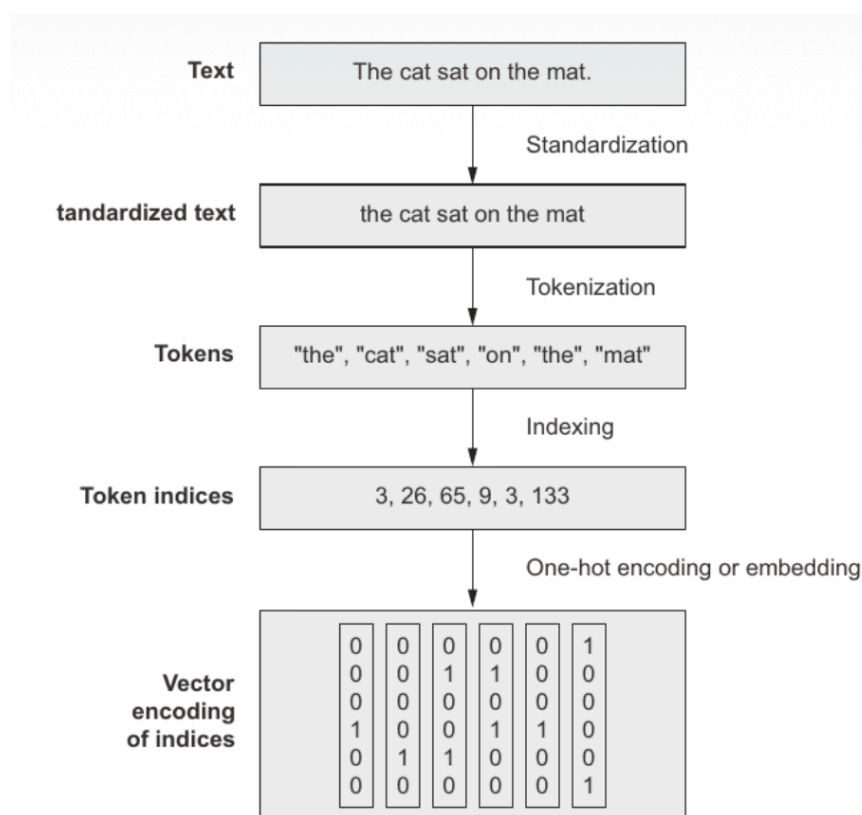
In this section, we only introduce the fundamental concepts needed to understand how neural networks process text. Each of these topics could easily fill an entire NLP course on its own, so our goal is to understand the core ideas rather than every implementation detail.

1. Introduction to Textual Data

Before a neural network can understand language, it must first convert text into numbers. Unlike humans, a neural network cannot directly process words or characters—it only accepts **numerical tensors** as input.

Almost every NLP pipeline follows four fundamental preprocessing stages:

1. **Standardization**
2. **Tokenization**
3. **Indexing**
4. **Vector Encoding**



From Raw Texts to Vectors

Each stage determines what information the model keeps and what information it ignores. Even small preprocessing decisions can significantly influence how easily the model learns and how well it generalizes to unseen text.

1.1. Standardization

The first step is to remove differences that are unlikely to carry useful meaning, preventing the model from wasting its capacity learning unnecessary distinctions.

Common standardization operations include:

- Converting all text to lowercase.
- Removing punctuation.
- Normalizing Unicode characters.
- Applying stemming or lemmatization.

→ For example, "The Cat sat on the mat." becomes "the cat sat on the mat"



A natural question is: **What about proper nouns?**

Lowercasing indeed removes the distinction between **"Apple"** (the company) and **"apple"** (the fruit). This is a trade-off.

- For many text classification tasks, the loss is minimal while the vocabulary becomes much smaller, making learning easier.
- However, for applications such as named entity recognition or machine translation, preserving capitalization may be important.

Modern preprocessing therefore depends on the task rather than following one universal rule.

- **Unicode normalization** removes different representations of the same character.

→ For example, México may be normalized to mexico

- Similarly, stemming and lemmatization reduce different forms of a word to a common base so that related words are treated similarly.

→ For example, `staring, stared, stares` may all be reduced to `stare`

Although stemming is computationally simpler, lemmatization generally preserves linguistic meaning more accurately.

1.2. Tokenization

After standardization, the text is divided into smaller units called **tokens**, which become the basic building blocks processed by the neural network.

- The simplest approach is **word-level tokenization** → E.g., `" the cat sat "` becomes `"the"`, `"cat"`, `"sat"`
- Sometimes, individual words do not capture enough context. Instead, we can group consecutive words into **n-grams** → For example, `" the cat sat "` produces the bigrams `"the cat"`, `"cat sat"`
- Modern NLP models often use **subword tokenization**, which splits words into meaningful fragments rather than entire words.
 - This approach is especially useful when the vocabulary is extremely large, when rare words appear frequently, or when spelling mistakes are common.
 - Some applications even tokenize at the character level, allowing the model to process completely unseen words.

1.3. Vocabulary and Indexing

Once the training text has been tokenized, the model builds a **vocabulary**, assigning every known token a unique integer.

→ For example, `the → 3` ; `cat → 26` ; `sat → 65`

Neural networks never work directly with the words themselves—they only receive these integer IDs. In practice, vocabularies are usually limited to the **20,000–30,000** most frequent tokens. This greatly reduces memory usage while still covering the overwhelming majority of words encountered in real-world text.

Words that appear too infrequently—or never appeared during training—are replaced with a special token: `[UNK]` which stands for **Unknown**. Although some information is lost, this allows the model to continue processing sentences instead of failing whenever it encounters a new word.

1.4. Text Vectorization

Fortunately, we rarely implement these preprocessing steps manually.

- In TensorFlow, the `TextVectorization` layer combines several stages into a single component. By default, it converts text to lowercase, removes punctuation, splits text into tokens, builds the vocabulary, and converts text into numerical vectors.
- The layer supports several output formats, including:
 - `int` — sequence of token IDs.
 - `multi_hot` — indicates whether each vocabulary word appears.
 - `count` — stores the frequency of each token.
 - `tf_idf` — weights words according to both frequency and rarity.

Before using the layer, we call `adapt(train_texts)` which scans the training corpus and learns the vocabulary automatically.

1.5. Encoding and Decoding

Consider the sentence: `I write, rewrite, and still rewrite again.`

→ After vectorization, it may become something like `[7, 3, 5, 9, 1, 5, 10]`

These integers—not the original words—are what the neural network actually receives.

Later, we can decode these integers back into readable text using the learned vocabulary. However, any word that was not included in the vocabulary is reconstructed as `[UNK]`

This placeholder serves an important purpose.

- A neural network can only assign embeddings to words it has learned during training.

- When an unseen word appears, mapping it to **[UNK]** allows the model to continue processing the sentence instead of encountering an undefined input.

The key takeaway is that **preprocessing is not merely a technical convenience—it is part of the model itself**. The choices made during preprocessing determine the information available to the neural network and can have a significant impact on the model's final performance.

2. Representing Groups of Words

So far, we have mostly talked about representing **a single word**. But in real-world Natural Language Processing (NLP), we rarely analyze words in isolation. We usually work with **sentences, paragraphs, articles, or even entire books**. Regardless of the length, the underlying question remains the same:

How can we represent a group of words so that a machine can understand them?

Before answering that, we need to ask another important question:

Is textual data order-dependent, just like a time series?

Almost always, **yes**. The meaning of a sentence often depends not only on **which words appear**, but also **where they appear**. A translator, for example, cannot simply translate one word at a time. It must understand the surrounding context before deciding what each word actually means.

Consider the word **book** in these three sentences:

- **I book a train ticket.** (*book = reserve*)
- **I read a book about trains.** (*book = a written publication*)
- **I train a model to read a book.** (*book still means a publication, but "train" now means teaching an AI model instead of practicing a skill.*)

The same word can represent completely different ideas depending on the surrounding words. This is why representing text is much more challenging than representing numbers.

Over the years, researchers have proposed many different techniques. Some completely ignore word order, while others try to preserve it as much as possible.

2.1. Bag of Words (BoW)

One of the earliest and simplest techniques is called the **Bag of Words (BoW)** model.

The basic idea is surprisingly simple: **ignore the order of the words completely**. We only care about **which words appear** in the document.

For example, consider the sentence: " The cat sat on the mat"

- Using **Unigrams** (single words), we split the sentence into individual tokens:

```
the | cat | sat | on | the | mat
```

- Suppose our vocabulary is: [the, cat, dog, sat, on, mat]. The sentence becomes a **multi-hot vector**: [1, 1, 0, 1, 1, 1]
 - A value of **1** means the word appears at least once.
 - A value of **0** means it does not appear.

Notice something interesting. The sentences: " The cat sat on the mat" and " The mat sat on the cat" produce exactly the same Bag-of-Words representation, even though they clearly mean different things. BoW completely loses word order.

- **Adding N-grams**

To preserve a little bit of local word order, we can use **N-grams**. Instead of treating words individually, we combine consecutive words.

For example:

- **Unigram**: " the", "cat", "sat", "on", "mat"
- **Bigram**: " the cat", "cat sat", "sat on", "on the", "the mat"

Bigrams provide some local context while remaining relatively simple.

In TensorFlow, this can be done using:

- `TextVectorization(output_mode="multi_hot")` or `TextVectorization(ngrams=2, output_mode="multi_hot")`

2.2. TF-IDF Weighting

Bag of Words treats every word equally. But should the words **the**, **is**, and **of** be just as important as **earthquake**, **fraud**, or **cancer**? Probably not.

This leads to one of the oldest and most effective improvements to Bag of Words: **TF-IDF (Term Frequency–Inverse Document Frequency)**.

The idea is intuitive.

- A word appearing many times in one document is probably important.
- A word appearing in almost every document is usually less informative.

For example, imagine two news articles.

- **Article A:** "Python is popular. Python is easy to learn."
- **Article B:** "Football is popular around the world."

The word "**popular**" appears in both articles, so it tells us very little about either one. However, the word **Python** appears repeatedly only in Article A, making it much more useful for identifying the topic.

TF-IDF combines these two ideas:

$$\text{TF-IDF} = tf \times \log \left(\frac{N}{df} \right)$$

where:

- **TF (Term Frequency):** How often a word appears in one document.
- **DF (Document Frequency):** How many documents contain that word.
- **N:** Total number of documents.

Words that are frequent in one document but rare across the collection receive the highest weights.

2.3. Sequence Models

Traditional methods such as Bag of Words and TF-IDF work well for many tasks, but they ignore much of the sentence structure.

Instead of manually designing features like unigrams or bigrams, modern NLP often lets the model learn directly from the sequence.

The input is represented as a sequence of token IDs: $x = (x_1, x_2, \dots, x_T)$

→ For example: “ The cat sat” might become [15, 83, 42]

Each integer is then converted into a vector, and the model processes them **in order**. Unlike Bag of Words, the model knows that **cat** comes before **sat**. Models such as: RNN; 1D Convolutional Networks; Transformers learn these sequential relationships automatically.

Instead of manually engineering features, we simply provide the sequence and let the model discover which patterns are important.

2.4. Word Embeddings

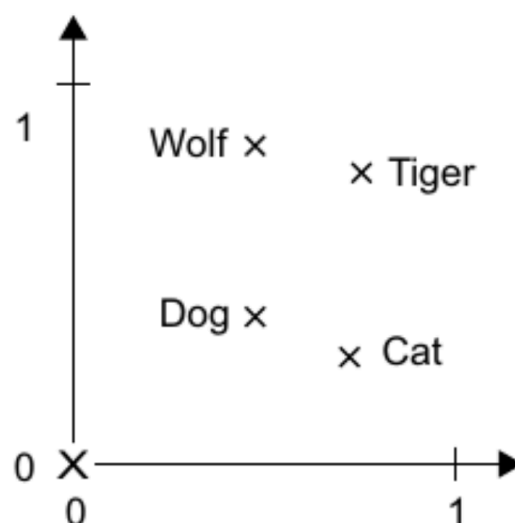
So far, words have mostly been represented using one-hot vectors. The problem is that one-hot vectors treat every word as completely unrelated.

For example: Dog = [0 1 0 0 0] and Wolf = [0 0 1 0 0]

Although dogs and wolves are closely related, their vectors have nothing in common.

Word embeddings solve this problem. Instead of sparse one-hot vectors, each word is represented by a **dense vector** with far fewer dimensions.

More importantly, words with similar meanings tend to appear close together in the embedding space. Imagine plotting words on a coordinate plane.



- **Dog** and **Wolf** appear close together.
- **Cat** and **Tiger** also appear close together.
- Similarly, **Chicken** and **Rooster** form another nearby pair.

The model discovers these relationships automatically because these words often appear in similar contexts.

Even more interesting, directions in the embedding space often capture semantic relationships such as: singular ↔ plural; male ↔ female; country ↔ capital

Embeddings can be learned during training, or loaded from pretrained models such as **Word2Vec**, **GloVe**, or modern **Sentence Transformers**.

In TensorFlow, embeddings are created using the **Embedding** layer.

2.5. Padding and Masking

One final practical problem remains. Sentences rarely have the same length.

→ For example: *"I love AI"* contains only three words, while *"I really love learning Natural Language Processing"* contains many more.

Neural networks process data in **batches**, and every sequence inside a batch must have the same length.

The simplest solution is **padding**. We add a special token (usually **0**) to shorter sequences until every sentence has the same length.

→ For example: *"I love AI"* becomes `[I, love, AI, 0, 0, 0]`

while *"I really love learning NLP"* might already have six tokens and therefore requires no padding.

However, these added zeros do not contain any information. If an RNN processes them like ordinary words, they may introduce unnecessary noise.

To solve this, we use **masking**. A mask tells the model: ***"These padding tokens are not real words. Ignore them during computation."***

As a result, the model focuses only on the meaningful parts of each sentence while safely ignoring the added padding.

3. Implementation using Python

Now that we understand the major ways of representing text, it is time to implement them ourselves.

Throughout this section, we will continue using the **IMDB Movie Review** dataset. Although our final goal is still **binary sentiment classification**, the focus is no longer on the classifier itself. Instead, we want to answer a different question:

How do we transform raw English text into something that a neural network can actually understand?

Unlike tabular datasets, a movie review is simply a sequence of characters. Models such as Logistic Regression, Decision Trees, or even Neural Networks cannot directly process strings like: *"This movie was absolutely amazing!"*

Before training any model, we must first convert these reviews into numerical representations. This preprocessing pipeline is one of the most important components of every NLP project, and the functions we build here will later be reused when training **Transformers** and **Sequence-to-Sequence (Seq2Seq)** models.

3.1. Loading the IMDB Dataset

Fortunately, TensorFlow already provides a convenient way to load the IMDB dataset from disk.

Recall that the dataset contains:

- **25,000** training reviews
- **25,000** testing reviews
- Balanced positive and negative samples
- Each review stored as a separate text file

Each text file is one movie review, while the parent folder (`pos` or `neg`) provides its sentiment label.

TensorFlow can automatically scan this directory and create a `tf.data.Dataset`.

```
import tensorflow as tf
from tensorflow import keras
```

```
train_dir = "./aclImdb_v1/aclImdb/train"
test_dir = "./aclImdb_v1/aclImdb/test"
```

Note: You may need to download and extract the dataset before loading it!!!

Dataset: "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"

```
class_names = ['neg', 'pos']

train_ds = keras.utils.text_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="training",
    seed=42,
    batch_size=32,
)

val_ds = keras.utils.text_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="validation",
    seed=42,
    batch_size=32,
)

test_ds = keras.utils.text_dataset_from_directory(
    test_dir,
    batch_size=32,
)
```

Note that the IMDB dataset used here has another folder called 'unsup' which contains unsupervised data.

We will ignore that folder by explicitly defining the class_names argument.

Each batch contains two tensors: Review text and Sentiment label (0, 1, or 2). For example,

```
for reviews, labels in train_ds.take(1):
    for i in range(3):
        review = reviews.numpy()[i].decode("utf-8")
        # Insert a newline after every period followed by a
space
        review = review.replace(". ", ".\n")
        print("Review:")
        print(review)
        print("+ Label:", labels.numpy()[i])
        print("-" * 80)
```

At this point, the reviews are still plain text. Before we can build a vocabulary or feed them into a neural network, we should clean and standardize them.

3.2. Text Standardization and Tokenization

Real-world text is messy. Consider these two reviews: " This movie is AMAZING!!!
" and

"this movie is amazing"

Humans immediately recognize them as identical, but a computer initially sees completely different strings because of uppercase letters and punctuation.

Therefore, the first preprocessing step is **text standardization**. Typically, this includes converting everything to lowercase, removing punctuation, optionally removing accents, collapsing multiple spaces into one.

After cleaning, we perform **tokenization**, which simply splits a sentence into individual words.

The following helper functions perform these preprocessing steps.

```
import re
import string
import unicodedata
```

```

def remove_accents(text):
    normalized = unicodedata.normalize("NFD", text)
    return "".join(ch for ch in normalized
                    if unicodedata.category(ch) != "Mn")

def standardize_text(text, strip_accents=False):
    text = text.lower()

    if strip_accents:
        text = remove_accents(text)

    text = re.sub(f"[{re.escape(string.punctuation)}]", "",
text, flags=re.IGNORECASE)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def tokenize_words(text):
    return standardize_text(text).split()

```

Let us test them on several movie reviews.

```
# Test on a sample review
sample_review = "This movie was AMAZING! I loved it. The acting was great, and the plot was engaging. Highly recommended!"

for sentence in sample_review.split('.'):
    print("Original :", sentence)
    print("Cleaned  :", standardize_text(sentence))
    print("Tokens   :", tokenize_words(sentence))
    print()
```

Original : This movie was AMAZING! I loved it

Cleaned : this movie was amazing i loved it

Tokens : ['this', 'movie', 'was', 'amazing', 'i', 'loved', 'it']

Original : The acting was great, and the plot was engaging

Cleaned : the acting was great and the plot was engaging

Tokens : ['the', 'acting', 'was', 'great', 'and', 'the', 'plot', 'was', 'engaging']

Original : Highly recommend!

Cleaned : highly recommend

Tokens : ['highly', 'recommend']

Now every review follows the same format, making later processing much easier.

3.3. Building a Vocabulary and Encoding Text

Neural networks cannot understand words. Instead, each word must be assigned a unique integer. For example, movie → 2; great → 3; terrible → 4; acting → 5

This mapping is called a **vocabulary**. We begin with two special tokens: **0** → padding; **1** → unknown word (`[UNK]`)

Every other word is assigned a new index as it is discovered in the training data.

```
vocab = {"": 0, "[UNK]": 1, }
```



```

for reviews, _ in train_ds:

    for review in reviews.numpy():
        text = review.decode("utf-8")

        for token in tokenize_words(text):
            if token not in vocab:
                vocab[token] = len(vocab)

inverse_vocab = {
    idx: token
    for token, idx in vocab.items()
}

```

Next, we create two helper functions. The first converts a sentence into token indices. The second reconstructs a sentence from those indices.

```

def encode_text(text, vocab):
    return [vocab.get(token, vocab["[UNK]"])
            for token in tokenize_words(text)]

def decode_ids(ids, inverse_vocab):
    return " ".join(inverse_vocab[idx]
                    for idx in ids if idx != 0)

```

We can now test our vocabulary.

```

test_sentences = [
    "This movie was AMAZING! I loved it. The acting was gre
at, and the plot was engaging. Highly recommend!",
    "C'était un film incroyable! J'ai adoré. Le jeu d'acteu
r était excellent et l'intrigue captivante. Je recommande v
ivement!",
    "The cinematography was breathtaking, and the soundtrac
k perfectly complemented the scenes.",
    "I found the pacing of the movie to be a bit slow, but
the character development made up for it.",
    "The ending was unexpected and left me thinking about t
he story long after it finished."
]

sample = " ".join(test_sentences)
encoded = encode_text(sample, vocab)

print("Vocabulary size:", len(vocab))
print("Original :", sample)
print("Encoded :", encoded)
print("Decoded :", decode_ids(encoded, inverse_vocab))

```

Vocabulary size: 207350

Original : This movie was AMAZING! I loved it. The acting was great, and the plot was engaging. Highly recommend! C'était un film incroyable! J'ai adoré. Le jeu d'acteur était excellent et l'intrigue captivante. Je recommande vivement! The cinematography was breathtaking, and the soundtrack perfectly complemented the scenes. I found the pacing of the movie to be a bit slow, but the character development made up for it. The ending was unexpected and left me thinking about the story long after it finished.

Encoded : [8, 168, 9, 1579, 5, 953, 15, 38, 477, 9, 445, 44, 38, 258, 9, 972, 233, 121, 1, 3394, 75, 1, 63811, 1, 7796, 97623, 1, 175437, 812, 5416, 1, 1, 45478, 1, 31957, 38, 3182, 9, 9890, 44, 38, 3881, 1710, 40238, 38, 621, 5, 794, 38, 901, 31, 38, 168, 2, 3, 79, 852, 1559, 16, 38, 747, 4246, 419, 663, 36, 15, 38, 362, 9, 9301, 44, 640, 37, 2245, 413, 38, 434, 954, 528, 15, 8185]

Decoded : this movie was amazing i loved it the acting was great and the plot was engaging highly recommend [UNK] un film [UNK] jai [UNK] le jeu [UNK] était excellent et [UNK] [UNK] je [UNK] vivement the cinematography was breathtaking and the soundtrack perfectly complemented the scenes i found the pacing of the movie to be a bit slow but the character

development made up for it the ending was unexpected and left me thinking about the story long after it finished

Here we introduce some French movie reviews. Which apparently cannot appear in an English IMDB dataset

Unknown words that never appeared during training automatically become `[UNK]`. This integer representation is the foundation of nearly every modern NLP model.

3.4. TensorFlow's TextVectorization Layer

Building our own vocabulary is an excellent exercise because it shows what happens behind the scenes. However, for real projects, manually managing vocabularies quickly becomes inconvenient.

TensorFlow provides a layer called **TextVectorization** that automates almost everything:

- Building the vocabulary
- Assigning integer IDs
- Computing TF-IDF statistics
- Generating Bag-of-Words representations
- Creating N-grams

Instead of writing dozens of preprocessing functions ourselves, we simply let the layer **learn** the vocabulary from the training data.

The learning process is performed using the `adapt()` function. Think of `adapt()` as the text equivalent of fitting a scaler. Instead of learning the mean and standard deviation, it learns the words that appear in the dataset.

One useful feature of `TextVectorization` is its different **output modes**. The same sentence can be represented in completely different ways depending on the task.

Suppose we have the sentence: `" This movie was surprisingly good"`. We can ask TensorFlow to convert it into: **Integer token IDs; Multi-hot vectors; Count vectors; TF-IDF vectors**

The following example demonstrates each representation.

```
import tensorflow as tf
from tensorflow.keras.layers import TextVectorization

# Extract only review texts
train_text = train_ds.map(lambda x, y: x)

sample_batch = tf.constant(["this movie was surprisingly go
od"])

vectorizers = {
    "Integer Sequence":
        TextVectorization(
            max_tokens=20000,
            output_mode="int",
            output_sequence_length=10
        ),

    "Multi-Hot":
        TextVectorization(
            max_tokens=20000,
            output_mode="multi_hot"
        ),

    "Count":
        TextVectorization(
            max_tokens=20000,
            output_mode="count"
        ),

    "TF-IDF (Bigram)":
        TextVectorization(
            max_tokens=20000,
            output_mode="tf_idf",
            ngrams=2
        ),
}
```

```

for name, layer in vectorizers.items():

    layer.adapt(train_text)

    print("=" * 60)
    print(name)
    print(layer(sample_batch).numpy()[0])

```

Notice that the integer representation preserves word order, while the remaining three representations ignore the sequence and instead describe the sentence as a collection of features.

This is exactly why Bag-of-Words models are fast but lose contextual information.

3.5. Building a Bag-of-Words Sentiment Classifier

Now that TensorFlow can convert text into vectors, building a sentiment classifier becomes surprisingly straightforward.

Our first model is intentionally simple.

Instead of using Transformers or recurrent networks, we simply:

1. Convert each review into a Bag-of-Bigrams.
2. Feed the vector into a small neural network.
3. Predict whether the review is positive or negative.

Although this architecture looks primitive compared to modern language models, it remains a surprisingly strong baseline.

The entire preprocessing pipeline is embedded directly inside the neural network, allowing the model to accept **raw text** as input.

```

from tensorflow import keras
from tensorflow.keras import layers

bow_vectorizer = TextVectorization(max_tokens=20000,
                                   ngrams=2,

```

```

output_mode="multi_hot",)

bow_vectorizer.adapt(train_text)

inputs = keras.Input(shape=(), dtype=tf.string,)

x = bow_vectorizer(inputs)

x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.5)(x)

outputs = layers.Dense(1, activation="sigmoid")(x)

bow_model = keras.Model(inputs, outputs)
bow_model.compile(optimizer="adam", loss="binary_crossentropy",
                  metrics=["accuracy"])

```

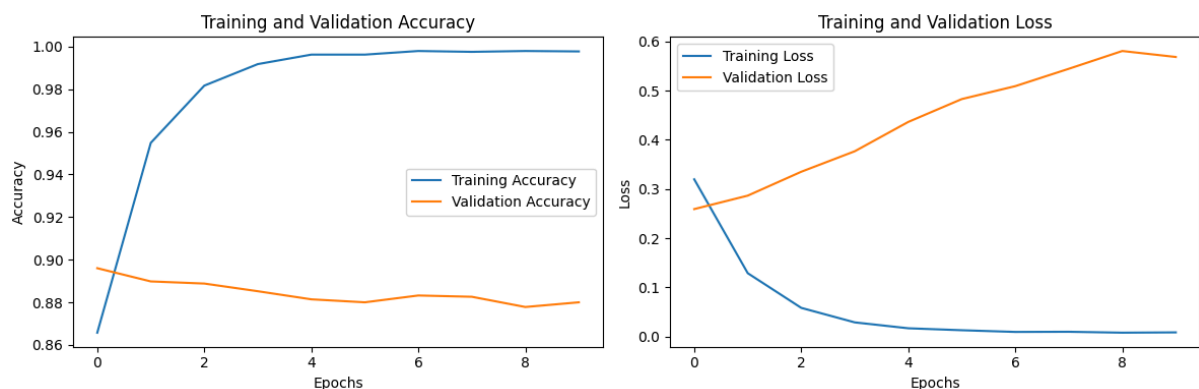
Training the model is equally simple.

```

history = bow_model.fit(train_ds, validation_data=val_ds, epochs=10)

```

Optionally, we can plot the accuracy and loss



After training, we evaluate the classifier on the testing set.

```
loss, accuracy = bow_model.evaluate(test_ds)
print("Test accuracy:", accuracy)
```

Test accuracy: 0.8845599889755249

Test loss: 0.5591595768928528

Finally, we can use the model to predict completely new reviews.

```
import numpy as np
import pandas as pd

new_reviews = np.array([
    "This movie was fantastic. But I hate the ending because it feels rushed. ",
    "The characters are well-developed and the plot is engaging. However, I believe a majority of them deserves more screen time. ",
    "Overall, I enjoyed it.",
], dtype=object)

probabilities = bow_model.predict(new_reviews, verbose=0).reshape(-1)

# 3. Create a summary DataFrame
results = pd.DataFrame({
    "Review": new_reviews,
    "Positive Probability": np.round(probabilities, 3),
    "Predicted Sentiment": ["Positive" if p > 0.5 else "Negative" for p in probabilities]
})

display(results)
```

Output: This review is Positive according to the model

Even though this model only uses Bag-of-Words features, it is capable of learning surprisingly strong sentiment patterns.

However, one limitation remains. The model still ignores the order of the words.

- The reviews: “ *I like this movie*”, and “**I don't like this movie**” share many of the same words, yet express opposite meanings.

To overcome this limitation, we need models that preserve the sequence of tokens.

3.6. Sequence Representation with Embeddings

Instead of converting a sentence into one giant feature vector, sequence models keep every token in its original order.

The first step is to represent each word as an **integer sequence**.

We can do this using another `TextVectorization` layer.

```
seq_vectorizer = TextVectorization(max_tokens=20000,
                                   output_mode="int",
                                   output_sequence_length=200)
seq_vectorizer.adapt(train_text)
```

Suppose a review begins with “ *This movie was fantastic*”. It might become something similar to `[15, 8, 62, 347, 0, 0, 0, ...]`. Each integer simply refers to one word inside the vocabulary.

However, integers themselves carry no semantic meaning. Word **347** is not inherently larger or more important than word **62**. This is where **Embeddings** become useful.

An Embedding layer transforms each integer into a dense vector that the neural network can learn during training.

```
embedding = layers.Embedding(input_dim=20000, output_dim=128,
                              mask_zero=True)
```

Now let us observe what happens.


```

for reviews, labels in train_ds.take(1):
    encoded = seq_vectorizer(reviews)
    embedded = embedding(encoded)
    mask = embedding.compute_mask(encoded)

    print("Encoded shape :", encoded.shape)
    print("Embedding shape :", embedded.shape)
    print("Mask shape :", mask.shape)

break

```

If the batch size is **32** and every review is padded to **200** words, the output becomes:

Encoded shape : (32, 200)

Embedding shape : (32, 200, 128)

Mask shape : (32, 200)

Notice what happened. Each integer has been replaced by a **128-dimensional vector**.

Instead of representing one review as a single Bag-of-Words vector, we now have an ordered sequence of learned word embeddings.

- The parameter `mask_zero=True` is equally important. Recall that shorter reviews are padded with zeros so that every sequence has the same length.
- Without masking, the model would mistakenly treat these padding tokens as actual words.
- With masking enabled, TensorFlow automatically tells later layers to ignore every position containing **0**.

3.7. Positional Embeddings

One major limitation of ordinary word embeddings is that they do not know where a word appears in the sentence.

Suppose we have these two sentences: `"The dog chased the cat"` and `"The cat chased the dog"`.

Both sentences contain exactly the same words. If we only look at their word embeddings, the model cannot distinguish which animal is doing the chasing.

Humans naturally understand word order, but a Transformer does not. Unlike an RNN, which processes one word after another, a Transformer reads the entire sentence simultaneously. This makes it extremely fast, but it also means that the model has no built-in notion of position.

To solve this problem, we introduce Positional Embeddings. The idea is surprisingly simple. Each token receives two vectors: Its word embedding and its position embedding

The final representation is simply the sum of these two vectors.

$$\text{Final Embedding} = \text{Word Embedding} + \text{Position Embedding}$$

For example,

Token	Position	Final Representation
the	0	Word(the) + Position(0)
dog	1	Word(dog) + Position(1)
chased	2	Word(chased) + Position(2)
the	3	Word(the) + Position(3)
cat	4	Word(cat) + Position(4)

Notice something interesting. Although `"the"` appears twice, its final representation is different because the position embeddings are different. This allows the Transformer to distinguish between identical words appearing at different locations.

The following custom layer implements this idea.

```
from tensorflow.keras import layers
import tensorflow as tf

class PositionalEmbedding(layers.Layer):
    def __init__(self, sequence_length, input_dim,
                  output_dim, **kwargs):
        super().__init__(**kwargs)

        self.positional_embeddings = tf.zeros(
            (sequence_length, input_dim, output_dim))
```

```

self.token_embeddings = layers.Embedding(
    input_dim=input_dim,
    output_dim=output_dim,
    mask_zero=True,
)

self.position_embeddings = layers.Embedding(
    input_dim=sequence_length,
    output_dim=output_dim,
)

def call(self, inputs):
    length = tf.shape(inputs)[1]
    positions = tf.range(start=0, limit=length, delta=
1,)

    token_embeddings = self.token_embeddings(inputs)
    position_embeddings = self.position_embeddings(posi
tions)

    return token_embeddings + position_embeddings

def compute_mask(self, inputs, mask=None):
    return self.token_embeddings.compute_mask(inputs)

```

We can now apply the layer to our encoded reviews.

```
pos_embedding = PositionalEmbedding(sequence_length=200,
                                     input_dim=20000,
                                     output_dim=128,)

for reviews, labels in train_ds.take(1):
    encoded = seq_vectorizer(reviews)
    output = pos_embedding(encoded)

    print(output.shape)
    break
```

The output is similar to (32, 200, 128)

Although the tensor shape remains unchanged, every embedding vector now contains both semantic information and positional information. This representation will become the input of our Transformer model in the next chapter.

3.8. Looking Ahead: Transformers and Seq2Seq Models

At this point, we have built almost the entire preprocessing pipeline used in modern Natural Language Processing.

Starting from raw movie reviews, we have learned how to: *clean text, tokenize words, build vocabularies, convert words into integer sequences, create Bag-of-Words features, generate TF-IDF vectors, learn dense word embeddings, preserve sentence order using positional embeddings.*

These steps are not specific to sentiment analysis. In fact, they are reused in almost every NLP application, including: Machine translation; Text summarization; Question answering; Chatbots; Large Language Models (LLMs)

The next models we study will build directly on top of this preprocessing pipeline.

- For Transformers, the positional embeddings we just created become the model's input before the self-attention mechanism.
- For Sequence-to-Sequence (Seq2Seq) models, both the encoder and decoder begin by transforming raw text into embeddings before generating

translated or summarized text.

In other words, although the architectures become much more sophisticated, the preprocessing pipeline remains remarkably similar.

This is one reason why learning text representation is so important—it serves as the foundation for nearly every modern NLP system.

Summary

In this section, we explored how raw text can be transformed into numerical representations for machine learning models.

- We introduced **Bag of Words**, **N-grams**, and **TF-IDF** as traditional techniques for representing documents, followed by **sequence representations** and **word embeddings**, which preserve word order and semantic relationships.
- We also discussed the importance of **padding** and **masking** for handling variable-length sequences.
- Finally, we implemented the complete preprocessing pipeline in TensorFlow, including **TextVectorization**, vocabulary construction, Bag-of-Words models, sequence encoding, and **positional embeddings**, which serve as the foundation for training **Transformers** and **Sequence-to-Sequence (Seq2Seq)** models in the following sections.

References

<https://towardsdatascience.com/deep-learning-techniques-for-text-classification-78d9dc40bf7c/>

<https://machinelearningmastery.com/best-practices-document-classification-deep-learning/>

Next Section:

 [17.1. The Transformer Models](#)